

Integración de Servicios Cloud GCP

Configuración, servicios Linux y proxy inverso

Semana 06 - Sesión 11

Rel Guzman



**Universidad
Tecnológica
del Perú**

Python y configuración

Python lee un archivo .env

Una **variable de entorno** es un valor con nombre que un programa puede leer. `.env` guarda esos valores en texto.

La configuración

```
# .env  
APP_MESSAGE=Hola
```

`load_dotenv` carga el archivo.
`os.environ` recupera el valor por su nombre.

La respuesta

Hola

El programa

```
import os  
from pathlib import Path  
from dotenv import load_dotenv  
  
load_dotenv(  
    Path(__file__).with_name(".env")  
)  
print(os.environ["APP_MESSAGE"])
```

`Path` localiza `.env` junto al programa. Python necesita esta carga explícita.

Una **biblioteca** es código reutilizable. Un **entorno virtual** reúne las bibliotecas de un proyecto en su propia carpeta.

```
sudo apt update
sudo apt install python3-venv curl
cd ~/sesion11
python3 -m venv .venv
.venv/bin/python -m pip install \
    -r requirements.txt
cp .env.example .env
```

Qué hace cada herramienta

sudo: permisos de administrador.

apt: instala paquetes Ubuntu.

pip: instala bibliotecas Python.

cd: cambia de carpeta.

cp: copia archivos.

Ubuntu 24.04. Copia antes demo/ a ~/sesion11. ~ es tu carpeta personal. -m ejecuta un módulo, -r lee la lista y \ continúa la orden en la siguiente línea.

Un **servidor** espera solicitudes y devuelve respuestas. **HTTP** define ese intercambio y **TCP** transporta sus datos en orden.

.env del servidor

```
APP_HOST=127.0.0.1  
APP_PORT=8000  
APP_MESSAGE=Hola desde Sesion11
```

Dirección IP

127.0.0.1: esta misma máquina. Se llama **loopback**.

Puerto

8000: el punto de entrada al programa dentro de esa máquina.

app.py usa `http.server`, la biblioteca didáctica de Python. Lee estos valores y devuelve el mensaje. Código completo en `demo/app.py`. `.env` es texto sin cifrar.

Una petición, una respuesta

`curl` es un cliente HTTP de consola: pide una respuesta al servidor.

Terminal 1: el servidor

```
.venv/bin/python app.py
```

Queda esperando solicitudes. Ctrl+C lo detiene.

Terminal 2: el cliente

```
curl http://127.0.0.1:8000/
```

La ruta `/` pide el recurso inicial.

Resultado esperado

Hola desde Sesion11

Cambiar `APP_MESSAGE` requiere volver a ejecutar Python. La configuración se lee al iniciar. Una variable ya existente en el entorno tiene prioridad sobre `.env`.

Puertos y procesos

Quién está escuchando

Proceso

Un programa en ejecución.

Python está ejecutando `app.py`. Su **PID** es el número que lo identifica en Linux.

Socket

Un extremo de comunicación.

Un socket **en escucha** espera conexiones en una dirección y un puerto.

127.0.0.1:8000

La recepción está abierta. Falta identificar quién la abrió.

ss -ltnp, letra por letra

ss consulta los sockets que conoce Linux. -ltnp reúne cuatro opciones.

```
sudo ss -ltnp
```

-l Escuchas abiertas.

-n Direcciones y puertos numéricos.

-t Conexiones TCP.

-p Proceso asociado.

Equivale a `ss -l -t -n -p`. sudo permite ver procesos de otros usuarios. El comando consulta el estado; no abre puertos.

Fragmento ilustrativo de la salida

```
LISTEN ... 127.0.0.1:8000 ... ("python",pid=2451,fd=3)
```

LISTEN Hay un socket esperando conexiones.

127.0.0.1:8000 Recibe únicamente dentro de esta máquina.

pid=2451 Python es el proceso responsable.

`fd=3` identifica ese recurso abierto dentro del proceso. Los números variarán.

En la columna local, `0.0.0.0:8000` significaría todas las direcciones IPv4 de la máquina. Un **firewall**, un filtro de tráfico, aún puede impedir el acceso externo.

Servicios y registros

Un **servicio** es un programa administrado por el sistema, incluso sin una terminal abierta. Un **log** registra lo que ocurrió.

`systemd` El administrador.

`systemctl` El comando para hablar con él.

`journalctl` El comando para consultar sus logs.

El administrador `systemd` inicia con PID 1. Su componente `systemd-journald` recoge mensajes en el **journal**, el registro central.

Qué hace realmente systemctl

Una **unidad** es algo que systemd administra. Un archivo `.service` describe cómo ejecutar un servicio.

```
sudo systemctl start sesion11
```

- 1 `systemctl` envía la petición de inicio.
- 2 `systemd` lee la unidad y lanza Python.
- 3 `systemd` sigue supervisándolo cuando el comando termina.

`sesion11` equivale aquí a `sesion11.service`. `status` consulta su estado y `stop` pide detenerlo. Primero crearemos esa unidad.

journalctl: consultar lo ocurrido

Podemos empezar por los logs del sistema. Un **arranque** es el período que comienza al encender o reiniciar la máquina.

```
sudo journalctl -b -n 20 --no-pager
```

-b Solo el arranque actual.

--no-pager Salida directa, sin visor.

-n 20 Los últimos veinte registros.

-u Filtra por una unidad.

Cuando nuestro servicio esté creado

```
sudo journalctl -u sesion11 -f
```

-f sigue mostrando nuevos mensajes. Ctrl+C termina la consulta y el servicio continúa.

La receta del servicio

[Unit] describe el servicio. [Service] define su ejecución. [Install] indica cómo incorporarlo al arranque.

Fragmento de `sesion11.service`

```
[Service]
User=sesion11
WorkingDirectory=/opt/sesion11
ExecStart=/opt/sesion11/.venv/bin/python -u /
    opt/sesion11/app.py
Restart=on-failure
RestartSec=3
StandardOutput=journal
StandardError=journal
```

User: cuenta que lo ejecuta.

WorkingDirectory: carpeta de trabajo.

ExecStart: comando exacto. `-u` escribe los mensajes sin demora.

Restart: reintenta tras un fallo, esperando 3 segundos.

StandardOutput/Error: envían salida y errores al journal.

La unidad completa está en `demo/sesion11.service`. `ExecStart` es una sola línea. Python carga `/opt/sesion11/.env` al iniciar.

Creación e inicio del servicio

La cuenta `sesion11` debe existir y poder leer la aplicación instalada en `/opt/sesion11`. La guía incluye esa preparación.

```
# Desde ~/sesion11, con la preparacion terminada
sudo cp sesion11.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl start sesion11
sudo systemctl status sesion11 --no-pager
sudo systemctl enable sesion11
```

daemon-reload

Relee las definiciones de unidades.

Detén antes el Python manual con Ctrl+C para liberar 8000. `active (running)` confirma el proceso; `curl` comprueba que además responde.

start / enable

Inicia ahora / prepara futuros arranques.

La orden depende del cambio

En cada caso, `systemctl` comunica la acción al administrador.

Qué necesitas	Orden después de <code>sudo systemctl</code>
Releer <code>.env</code> o código Python	<code>restart sesion11</code>
Aplicar cambios al archivo <code>.service</code>	<code>daemon-reload</code> , luego <code>restart sesion11</code>
Detener ahora	<code>stop sesion11</code>
Quitar inicio automático y detener	<code>disable --now sesion11</code>

`restart` detiene e inicia. `reload` pide al programa releer su configuración si lo admite. Nuestro Python necesita reinicio. NGINX, que veremos a continuación, sí admite recarga.

NGINX y proxy inverso

Un intermediario delante de Python

NGINX es un servidor web. Como **proxy inverso**, recibe las peticiones del sitio y las entrega a la aplicación interna.

Cliente

Pide la página.

NGINX :80

Recibe y reenvía.

Python :8000

Genera la respuesta.

Conexión 1 El cliente se comunica con NGINX.

Conexión 2 NGINX se comunica con Python en 127.0.0.1.

La respuesta vuelve por esas conexiones. El navegador conserva su dirección original.

sites-available

Configuraciones disponibles.

Aquí guardamos el archivo original.

sites-enabled

Configuraciones habilitadas.

Aquí colocamos un **enlace simbólico**:
una referencia al original.

```
sudo apt install nginx
ls /etc/nginx/sites-available
ls -l /etc/nginx/sites-enabled
```

ls lista archivos; -l muestra detalles y destinos de enlaces. Son carpetas de la organización Ubuntu. nginx.conf las incorpora mediante include, la instrucción para leer otros archivos.

La configuración del proxy

server define un sitio y location / aplica sus reglas a las rutas que empiezan con /.

```
server {  
    listen 80;  
    server_name sesion11.local;  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

Entrada

listen: puerto.

server_name: nombre del sitio.

Destino

proxy_pass: aplicación interna.

Cabeceras

Datos que acompañan la petición.

proxy_set_header los ajusta.

\$host y \$remote_addr son variables de NGINX: nombre pedido e IP del cliente. Las llaves agrupan instrucciones y el punto y coma las termina.

`ln -s` crea el enlace. `nginx -t` valida la configuración. `reload` hace que NGINX la lea de nuevo.

```
cd ~/sesion11
sudo cp nginx-sesion11.conf /etc/nginx/sites-available/sesion11
sudo ln -s /etc/nginx/sites-available/sesion11 \
    /etc/nginx/sites-enabled/sesion11
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

La configuración se aplica al recargar

El enlace se aplica cuando NGINX relee su configuración.

Crea el enlace una sola vez. Continúa con la recarga únicamente si la validación es correcta.

`enable --now` habilita e inicia el servicio.

La cabecera HTTP `Host` indica el sitio pedido. `curl -H` la agrega y `-i` muestra las cabeceras de respuesta.

Directo a Python

```
curl http://127.0.0.1:8000/
```

A través de NGINX

```
curl -i -H "Host: sesion11.local" http://127.0.0.1/
```

Esperamos 200 OK y el mismo mensaje.

200 OK significa éxito. Si solo funciona el acceso directo, revisa NGINX y el nombre del sitio. Para probar desde fuera, usa la IP de la máquina y permite TCP 80 en sus reglas de red.

Captura de tráfico

Un **paquete** lleva datos de red. Una **interfaz** es el punto por el que pasan.
Un capturador observa esos paquetes.

tcpdump Captura sencilla desde la consola.

Wireshark Analiza paquetes con interfaz gráfica.

TShark Wireshark desde la consola, sin escritorio.

En Linux, `lo` es la interfaz interna o loopback. Ahí veremos el intercambio entre NGINX y Python.
Un **filtro** selecciona los paquetes que nos interesan.

La captura más sencilla: tcpdump

Terminal 1: observar

```
sudo apt install tcpdump  
sudo tcpdump -i lo -nn 'tcp port 8000'
```

-i lo

Interfaz interna.

-nn

IP y puertos numéricos.

tcp port 8000

Puerto de origen o destino.

Terminal 2: generar tráfico

```
curl -H "Host: sesion11.local" http://127.0.0.1/
```

La petición entra por 80, pero observamos el tramo interno en 8000. Ctrl+C detiene la captura. Sin peticiones, es normal que espere sin mostrar paquetes.

En TCP, un **puerto temporal** identifica al cliente. El servidor conserva su puerto de escucha.

```
127.0.0.1.53012 > 127.0.0.1.8000: Flags [S]  
127.0.0.1.8000 > 127.0.0.1.53012: Flags [S.]
```

Salida ilustrativa. > indica dirección. Las marcas S y S. solicitan y aceptan abrir la conexión. Después viaja HTTP.

Para leer el contenido

```
sudo tcpdump -i lo -nn -s 0 -A 'tcp port 8000'
```

-A muestra texto y -s 0 evita un límite de captura pequeño. Busca GET /, la solicitud de un recurso. **HTTPS** cifra el contenido: en ese tramo no verías el texto. Usa tráfico de laboratorio sin secretos.

TShark: Wireshark en la terminal

TShark captura paquetes y muestra un resumen interpretado de cada uno.

```
sudo apt install tshark  
sudo tshark -i lo -n -f "tcp port 8000"
```

- i lo** Escoge la interfaz interna.
- n** Conserva las direcciones numéricas.
- f** Introduce el filtro que decide qué capturar.

Genera otra petición con curl en una segunda terminal. Ctrl+C termina. Si la instalación pregunta por capturas sin administrador, puedes elegir No y usar `sudo` en este laboratorio.

pcapng es un formato de archivo para paquetes. `-w` guarda, `-r` lee y `-Y` elige qué mostrar al analizar.

Capturar, generar peticiones y detener con Ctrl+C

```
sudo tshark -i lo -f "tcp port 8000" \  
-w /tmp/sesion11.pcapng
```

Mostrar únicamente HTTP

```
sudo tshark -r /tmp/sesion11.pcapng -Y "http"
```

El filtro `-f` decide qué guardar; `-Y`, qué mostrar. Si no reconoce HTTP, agrega `-d tcp.port==8000,http` al leer: indica cómo interpretar ese puerto. También puedes abrir el archivo en Wireshark.

Dónde buscar cuando algo falla

La evidencia

La siguiente comprobación

No hay escucha en 8000

`systemctl status sesion11` y `journalctl -u sesion11`.

Python funciona, aparece otro sitio

Cabecera Host y enlace en `sites-enabled`.

NGINX devuelve 502

No obtuvo respuesta válida de Python. Revisa su escucha y `/var/log/nginx/error.log`.

Captura vacía

Genera una petición. Comprueba interfaz y filtro.

Código, preparación completa y comandos: `Sesion11/README.md` y `demo/`.

Referencias: `python-dotenv`, Python, `ss`, `systemctl`, `journalctl`, NGINX, `tcpdump` y TShark.